

IGrade – Elektroniczny dziennik

Spis treści

IGrade – Elektroniczny dziennik.....	1
Słowem wstępu	4
Wybrany fragment rzeczywistości	4
Technologie oraz biblioteki (back-end):	5
Spring Boot 3.2.6-SNAPSHOT	5
Spring Cloud 2023.0.1	5
Spring boot data jdbc	5
Spring boot data jpa	5
Spring boot jdbc.....	5
Spring boot validation	5
Spring cloud netflix eureka client/server	5
Spring boot security	5
Spring boot actuator	5
Spring cloud circuitbreaker resilience4j	5
Jwt from io.jsonwebtoken	5
Spring boot o2auth authorization server	5
Lombok	5
Spring boot web.....	5
Spring boot webflux	5
Spring boot data mongodb	5
Mysql connector j	5
For testing endpoints:	6
POSTMAN	6
Architektura microserwisów	6
Zdjęcie 1.1	6
Autoryzacja Bearer Token (JWT).....	8
GetMapping: me	10
Synchronizacja	10
Implementacja komunikacji w kodzie:.....	11

LOGIKA Circuit Breaker	11
Closed State (Stan Zamknięty):.....	11
Open State (Stan Otwarty):	12
Half-Open State (Stan Półotwarty):	12
Implementacja tej logiki wraz z konfiguracją w samym kodzie:	12
Struktura projektu	12
Aspekt baz danych	13
MongoDB:	13
Dla Postgres oraz MySQL używamy JPARepository, które wykonuje zapytania do bazy danych:	17
Repozytorium:	17
Requesty	19
Przykładowy record z requestem:	20
Przykładowy endpoint realizujący request:	20
DTO oraz Mapperzy:	21
StudentDTO:	21
StudentMapper:	22
Przykładowy endpoint z mapperem:	23
Podział mikroservisów, ze względu na bazy danych:	23

Słowem wstępu

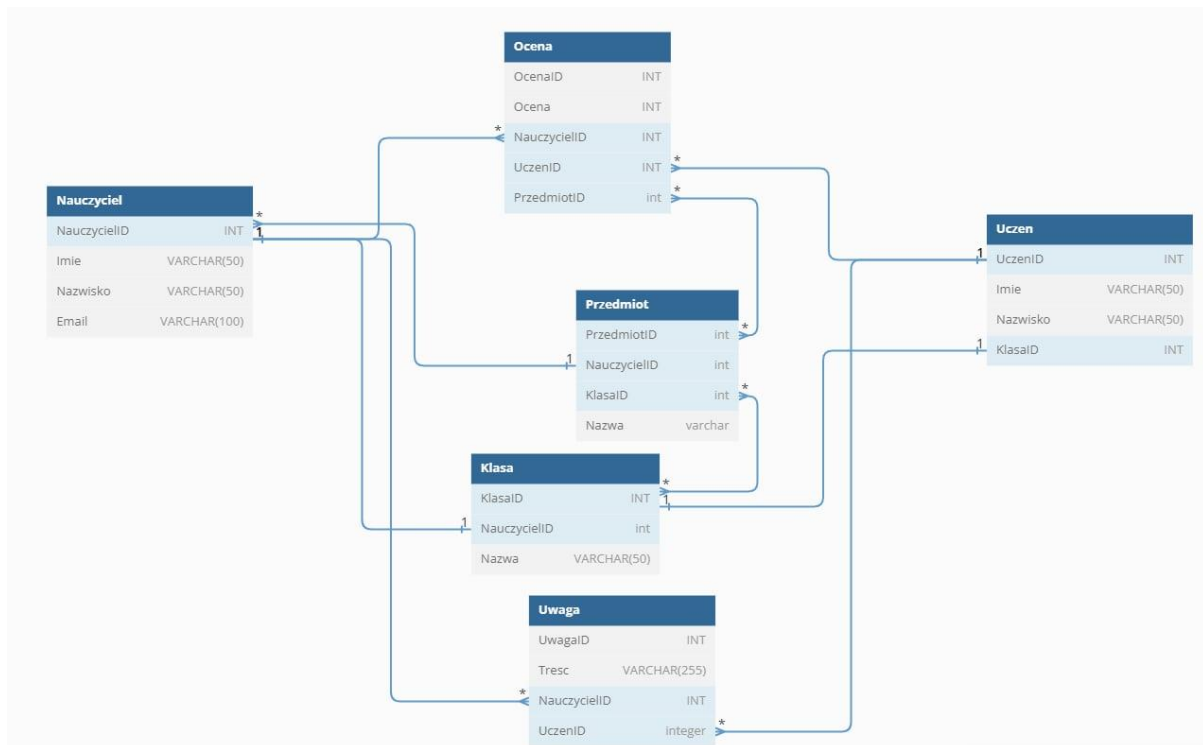
Projekt e-dziennika powstał na architekturze mikro service z wykorzystaniem Java Spring Boot.

Nasza zaprojektowana architektura wykorzystuje 3 różne bazy danych takie jak: MySQL, Postgres oraz MongoDB. Połączenia z bazami relacyjnymi takimi jak MySQL oraz Postgres odbywa się dzięki JDBC, MongoDB dla Spring Boot'a ma swój własny system repozytorium. Projekt powstał w celu pokazania możliwości mikroserwisów jak i zastosowaniu 3 różnych baz danych w których połączyliśmy dane swoim własnym systemem powiązań pomiędzy nimi. Dla przykładu relacja pomiędzy danymi z MySQL a danymi z MongoDB odbywa się dzięki wystawieniu odpowiedniej synchronizacji pomiędzy dwoma serwisami.

W projekcie chcieliśmy wykorzystać jak największą ilość zalet technologicznych. Użyliśmy między innymi netflix-eureka-server, który odpowiada za składowanie adresów docelowych pomiędzy API-GATEWAY a konkretnymi serwisami.

Wybrany fragment rzeczywistości

Wybraliśmy interpretację dziennika elektrycznego jako wybrany fragment rzeczywistości, diagram ERD przedstawia założenia powiązania tabel oraz relacje pomiędzy nimi. W celu poprawnego przedstawienia rzeczywistości stworzyliśmy podział na role takie jak STUDENT, NAUCZYCIEL oraz ADMIN. Gdzie student to uczeń szkoły, nauczyciel to oczywiście nauczyciel, admin odpowiada za dyrektora, vice-dyrektora, który może zarządzać większym obszarem oraz ma większe uprawnienia na samych endpointach takie jak pobranie wszystkich klas niezależnie czy jest ich wychowawcą.



Technologie oraz biblioteki (back-end):

Spring Boot 3.2.6-SNAPSHOT

Spring Cloud 2023.0.1

Spring boot data jdbc

Spring boot data jpa

Spring boot jdbc

Spring boot validation

Spring cloud netflix eureka client/server

Spring boot security

Spring boot actuator

Spring cloud circuitbreaker resilience4j

Jjwt from io.jsonwebtoken

Spring boot o2auth authorization server

Lombok

Spring boot web

Spring boot webflux

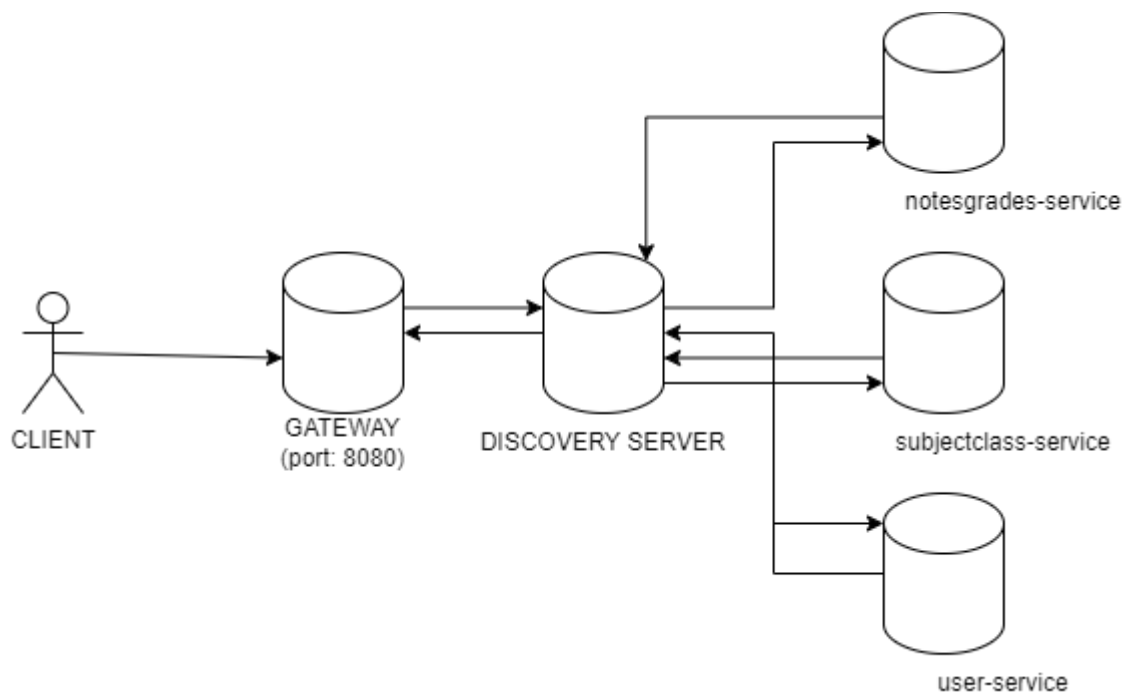
Spring boot data mongodb

Mysql connector j

For testing endpoints:

POSTMAN

Architektura microserwisów



Zdjęcie 1.1

Architektura została zaprojektowana w standardowy sposób jak na strukture mikro serwisów. Gateway to serwis który odpowiada za przyjmowanie wszystkich zapytań od klienta, następnie discovery-server(patrz zdjęcie 1.1), który posiada informacje na temat wszystkich mikroserwisów udziela dostęp dla klienta na wysunięte endpoint'y, które są odpowiednio zdeklarowane w gateway'u. Sam gateway odpowiada również za autoryzację tokenów JWT o czym więcej później. Dzięki takiemu stanowi jesteśmy w stanie dokładać w nieskończoność odpowiednich serwisów. Możemy na przykład na jednym discovery-server mieć składowane 10 różnych user-service, 5 różnych subjectclass-service oraz jeden notesgrades-service dzięki systemu loadbalance, discovery server będzie rozkładał zapytania na różne serwisy o tej samej nazwie. W samym gateway'u posiadamy konfiguracje jakie endpointy mają być przekazywane pod konkretne serwisy w architekturze.

```
#User service route
spring.cloud.gateway.mvc.routes[0].id = user-service
spring.cloud.gateway.mvc.routes[0].uri = lb://user-service
spring.cloud.gateway.mvc.routes[0].predicates[0]=Path=/api/v1/student/**,/api/v1/teacher/**,/api/v1/authentication/**

#NotesGrades service route
spring.cloud.gateway.mvc.routes[1].id = notesgrades-service
spring.cloud.gateway.mvc.routes[1].uri = lb://notesgrades-service
spring.cloud.gateway.mvc.routes[1].predicates[0]=Path=/api/v1/grade/**,/api/v1/note/**

#NotesGrades service route
spring.cloud.gateway.mvc.routes[4].id = subjectclass-service
spring.cloud.gateway.mvc.routes[4].uri = lb://subjectclass-service
spring.cloud.gateway.mvc.routes[4].predicates[0]=Path=/api/v1/class/**,/api/v1/subject/**

#Discovery Server Route
spring.cloud.gateway.mvc.routes[2].id = discovery-server
spring.cloud.gateway.mvc.routes[2].uri = http://localhost:8761
spring.cloud.gateway.mvc.routes[2].predicates[0]=Path=/eureka/web
spring.cloud.gateway.mvc.routes[2].filters[0]=SetPath=/

#Discover Server Static Resources Route
spring.cloud.gateway.mvc.routes[3].id = discovery-server-static
spring.cloud.gateway.mvc.routes[3].uri = http://localhost:8761
spring.cloud.gateway.mvc.routes[3].predicates[0]=Path=/eureka/**
```

Zdjęcie 1.2

Na przykładzie user-service, wszystkie endpoint'y, które trafiają na api-gateway pod adresem /api/v1/student, /api/v1/teacher, /api/v1/authentication są przekazywane do user-service pod adresem lb://user-service (patrz Zdjęcie 1.2), gdzie następnie eureka-netflix-server odpowiednio mapuje do konkretnych już adresów ze swojej puli serwisów. W sytuacji kiedy żaden serwis nie jest dostępny dla konkretnego routa oczywiście zakładamy, że serwis jest po prostu nie dostępny (więcej o tym w sync communication).

Żeby dany endpoint był wystawiony na api-gateway'u, dzięki autoryzacji oraz systemie security spring boot musimy go dodać do security configu, przykładowy config:

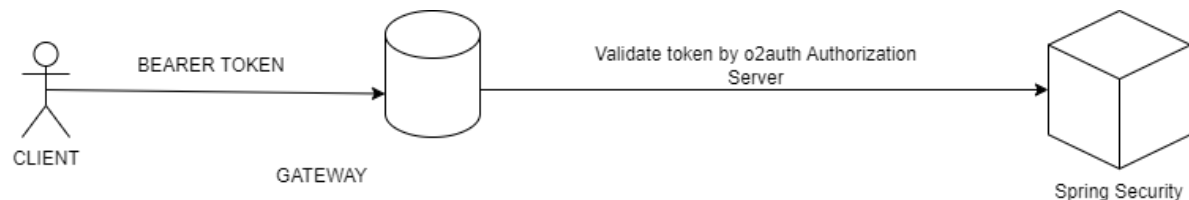
```
security.authorizeHttpRequests(
    auth -> auth.requestMatchers(POST, @"/api/v1/student/register").permitAll()
        .requestMatchers(POST, @"/api/v1/student/login").permitAll()
        .requestMatchers(POST, @"/api/v1/student/changePassword").authenticated()
        .requestMatchers(GET, @"/api/v1/student/firstname/**").authenticated()
        .requestMatchers(GET, @"/api/v1/student/lastname/").authenticated()
        .requestMatchers(GET, @"/api/v1/student/id/**").authenticated()
        .requestMatchers(POST, @"/api/v1/authentication/login").permitAll()
        .requestMatchers(GET, @"/api/v1/authentication/me").authenticated()
        .requestMatchers(GET, @"/api/v1/authentication/authenticate").authenticated()
```

Zdjęcie 1.3

Dzięki takiemu rozwiązaniu mamy pełną możliwość zarządzania, który endpoint w naszym systemie ma być zabezpieczony tokenem JWT, który endpoint wymaga dodatkowych uprawnień oraz który ma być dostępny dla wszystkich użytkowników.

Autoryzacja Bearer Token (JWT)

W ramach projektu nie używaliśmy żadnych zewnętrznych bibliotek dla autoryzacji. Używamy swojej własnej autoryzacji, która jest oparta o spring security, O2 Auth Server oraz tokeny JWT. Aby zcentralizować autoryzację, całość procesu jest na gateway'u, który to weryfikuje poprawność tokenów JWT.



Sama generacja odbywa się dzięki user-service, który posiada możliwości generowania tokenu JWT. Poprzez to rozwiązanie nie musieliśmy łączyć komunikacją gateway'a oraz user-service przekazując użytkowników. Na samym user-service posiadamy własny UserDetailsService:

```
@RequiredArgsConstructor
@Service
public class StudentTeacherDetailsService implements UserDetailsService {

    private final StudentService studentService;
    private final TeacherService teacherService;

    private final AdminProperties adminProperties;

    2 usages create test CzarnaWoda
    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        final Optional<Student> optionalStudent = studentService.getStudentByEmail(email);

        if(optionalStudent.isPresent()){
            final Student student = optionalStudent.get();

            return new User(student.getEmail(),
                student.getPassword(),
                student.isEnabled(),
                accountNonExpired: true,
                credentialsNonExpired: true,
                accountNonLocked: true,
                Collections.singleton(new SimpleGrantedAuthority( role: "STUDENT")))
        );
    }

    final Optional<Teacher> optionalTeacher = teacherService.getTeacherByEmail(email);
    if(optionalTeacher.isPresent()){
        final Teacher teacher = optionalTeacher.get();
```


Serwis mapuje użytkowników do Spring Security User'a, dzięki temu również wiemy kiedy loguje się student a kiedy loguje się nauczyciel. Następnie sprawdzamy czy login i hasło zgadzają się poprzez wykorzystanie wbudowanego systemu w spring security UserPasswordAuthenticationToken:

```
public ResponseEntity<HttpResponse> authenticate(UserLoginRequest userLoginRequest){
    try{
        final Authentication authentication = authenticationProvider.authenticate(
            new UsernamePasswordAuthenticationToken(userLoginRequest.email(), userLoginRequest.password())
        );

        final String token = tokenService.generateToken(authentication, userLoginRequest.remember());

        Optional<Student> optionalStudent = studentService.getStudentByEmail(authentication.getName());

        Map<?,?> data = null;
        if(optionalStudent.isPresent()){
            data = Map.of( k1: "user", studentMapper.mapStudentToStudentDTO(optionalStudent.get()));
        }else{
            Optional<Teacher> optionalTeacher = teacherService.getTeacherByEmail(authentication.getName());

            if(optionalTeacher.isPresent()) {
                data = Map.of( k1: "user", teacherMapper.mapTeacherToTeacherDTO(optionalTeacher.get()));
            }
        }
    }
}
```

Następnie jeżeli email oraz hasło się zgadzają generujemy token na podstawie kluczy RSA.

```
public String generateToken(Authentication authentication, boolean longToken){
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("self")
        .issuedAt(now())
        //TODO change only for test 60*60*60*5
        .expiresAt(now().plus(longToken ? 2*60*60 : 60*60*60*5, ChronoUnit.SECONDS))
        .subject(authentication.getName())
        .claim( name: "roles", authentication.getAuthorities().stream().map(GrantedAuthority::getAuthority).toList())
        .build();
    return this.jwtEncoder.encode(JwtEncoderParameters.from(claims)).getTokenValue();
}
```

Na samym user-service zostawiliśmy logikę sprawdzania tokenów, aby użytkownik był w stanie na podstawie samego tokenu pobrać informacje o sobie na endpoint /me.

GetMapping: me

```
@GetMapping("/me")
public ResponseEntity<HttpResponse> me(Authentication authentication){

    Optional<Student> optionalStudent = studentService.getStudentByEmail(authentication.getName());

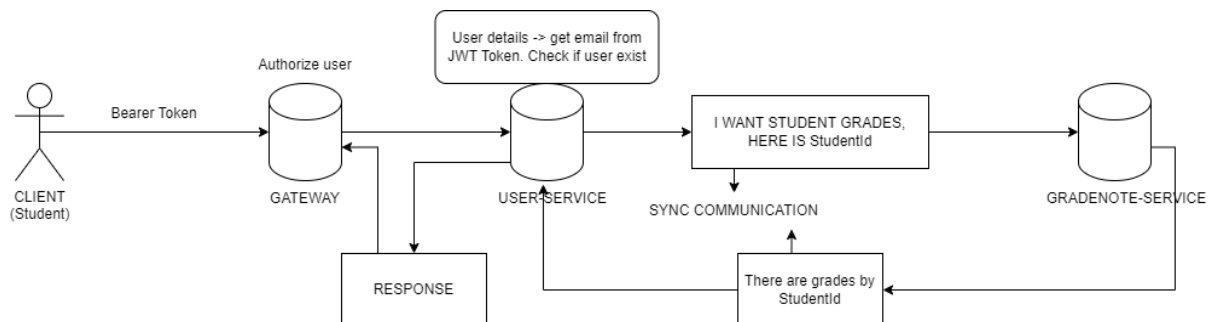
    if(optionalStudent.isPresent()){
        return ResponseEntity.status(OK).body(HttpResponse.builder()
            .message("Found user data")
            .data(Map.of("user", studentMapper.mapStudentToStudentAuthenticationDto(optionalStudent.get(), authentication)))
            .build());
    }else{
        Optional<Teacher> optionalTeacher = teacherService.getTeacherByEmail(authentication.getName());

        return optionalTeacher.map(teacher -> ResponseEntity.status(OK).body(HttpResponse.builder()
            .message("Found user data")
            .data(Map.of("user", teacherMapper.mapTeacherToTeacherAuthenticationDto(teacher, authentication)))
            .build()).orElseGet(() -> ResponseEntity.status(NOT_FOUND).body(
                HttpResponse.builder()
                    .message("User has not be found")
                    .build()
            ));
    }
}
```

Na wszystkich innych serwisach całą logikę autoryzacji za pomocą tokenu JWT przyjmuje na siebie api-gateway.

Synchronizacja

Bazowanie mikroservisów zawiera się nie tylko w samej strukturze, kluczową rolę w całości odgrywa komunikacja pomiędzy serwisami, w naszym przypadku nie chcemy podawać ocen ucznia, który być może nie jest zalogowany na konkretne konto. Aby uniknąć potencjalnego wycieku danych musieliśmy stworzyć komunikację, która wygląda jak na przedstawionym zdjęciu.



Jest to oczywiście przykład komunikacji, która została zaimplementowana.

Klient w naszym przypadku student, chce informacji na temat swoich ocen, aby nie wierzyć na słowo klientowi oraz uniknąć przypadku gdzie request zostanie po prostu zmanipulowany używamy autoryzacji za pomocą tokenu JWT, następnie po zweryfikowaniu sygnatury przez API-GATEWAY, przesyłamy request dalej na USER-SERVICE, który wydobywa z tokenu JWT email studenta. Na obu tych serwisach jest zaimplementowana logika dekompozycji tokenu JWT, dzięki temu mamy możliwość sprawdzenia czy konkretny email znajduje się w bazie danych mikroservisu UserService. Po sprawdzeniu tokenu przesyłamy zapytanie na zamknięty end-point, który nie jest wystawiony na zewnątrz. Pytamy u źródła (gradenote-service) o oceny

studenta z danym id, endpoint wyrzuca nam wszystkie oceny, które są przypisane do danego studenta. Sama odpowiedź jest mapowana przez USER-SERVICE, więc mamy pewność, że zapytanie wypytywa od właściciela ocen a nie jest przypadkowym requestem.

Implementacja komunikacji w kodzie:

```
public GradeDto[] getStudentGrades(long id) {  
    return WebClientBuilder.build().get() RequestHeadersUriSpec<capture of ?>  
        .uri( uri: "http://notesgrades-service/api/v1/grade/student/" + id) capture of ?  
        .retrieve() ResponseSpec  
        .bodyToMono(GradeDto[].class) Mono<GradeDto[]>  
        .block();  
}
```

Endpoint:

```
@GetMapping("/{grades}")  
@CircuitBreaker(name = "grades", fallbackMethod = "fallBackMethod")  
public ResponseEntity<HttpResponse> getGrades(Authentication authentication){  
  
    final Optional<Student> student = studentService.getStudentByEmail(authentication.getName());  
  
    return student.<=>map(value -> ResponseEntity.status(OK).body(HttpResponse.builder() HttpResponseBuilder<capture of ?, capture of ?>  
        .httpStatus(OK) capture of ?  
        .statusCode(OK.value())  
        .message("Student grades")  
        .data(Map.of( ki: "grades", studentService.getStudentGrades(value.getId()))  
        .build()))).orElseGet(() -> ResponseEntity.status(BAD_REQUEST).body(HttpResponse.builder().httpStatus(BAD_REQUEST)  
        .statusCode(BAD_REQUEST.value())  
        .message("Student doesn't have any grades").build()));  
}
```

Gdy logika została zaimplementowana powstał problem kluczowy.

Wyobrażając sobie sytuację, że aktualnie trwają prace nad NOTEGRADE-SERVICE co w przypadku gdy nie ma żadnej dostępnej instancji tego serwisu?

LOGIKA Circuit Breaker

Circuit breaker to wzorec projektowy, który ma na celu ochronę systemów przed przeciążeniem oraz umożliwienie ich szybszej reakcji na awarie. W kontekście mikroserwisów Spring Cloud, circuit breaker pozwala na zarządzanie interakcjami pomiędzy mikroserwisami w taki sposób, aby uniknąć kaskadowych awarii w całym systemie.

Jak działa circuit breaker w Spring Cloud:

Closed State (Stan Zamknięty):

W normalnych warunkach, gdy nie ma problemów z połączeniami, circuit breaker znajduje się w stanie zamkniętym. Wszystkie żądania przechodzą przez circuit breaker i są kierowane do docelowego mikroserwisu.

Open State (Stan Otwarty):

Gdy liczba nieudanych prób (np. timeouty, błędy sieci) przekroczy określony próg, circuit breaker przechodzi w stan otwarty. W tym stanie wszystkie żądania są natychmiast odrzucane, co zapobiega przeciążeniu docelowego serwisu oraz szybszemu wykryciu problemu.

Half-Open State (Stan Półotwarty):

Po pewnym czasie (określonym jako czas oczekiwania) circuit breaker przechodzi w stan półotwarty. W tym stanie przepuszczane są pojedyncze żądania testowe, aby sprawdzić, czy problem został rozwiązany. Jeśli testowe żądanie zakończy się sukcesem, circuit breaker wraca do stanu zamkniętego. Jeśli natomiast żądanie nadal kończy się błędem, circuit breaker wraca do stanu otwartego.

Dzięki takiemu rozwiązaniu jesteśmy w stanie kontrolować czy możemy pobrać np. Oceny z gradenote-service. Gdy nie jesteśmy wyrzucamy metode fallback, która wypisuje odpowiedni komunikat.

Implementacja tej logiki wraz z konfiguracją w samym kodzie:

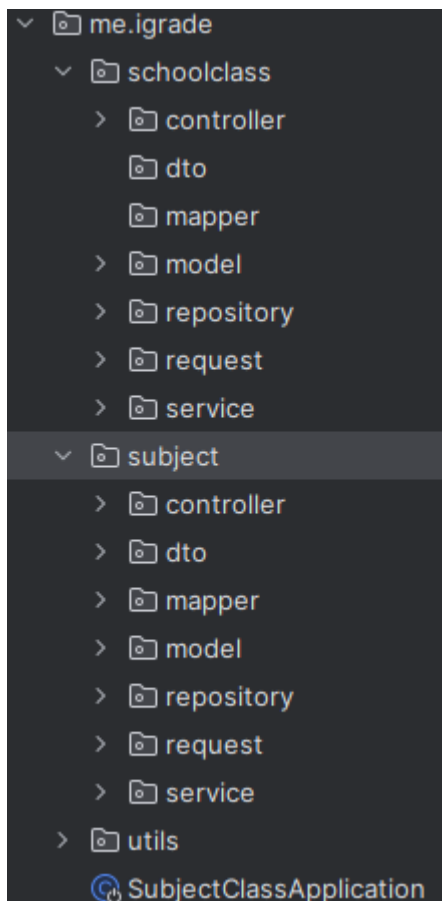
```
@GetMapping("/grades")
@CircuitBreaker(name = "grades", fallbackMethod = "fallBackMethod")
public ResponseEntity<HttpResponse> getGrades(Authentication authentication){

    public ResponseEntity<HttpResponse> fallBackMethod(Authentication authentication, RuntimeException runtimeException){
        return ResponseEntity.status(BAD_REQUEST).body(HttpResponse.builder()
            .httpStatus(BAD_REQUEST)
            .statusCode(BAD_REQUEST.value())
            .message("Oops! Aktualnie nie możemy pobrać informacji na temat ocen oraz uwag!")
            .build());
    }

#Resilience 4j properties
resilience4j.circuitbreaker.instances.grades.register-health-indicator=true
resilience4j.circuitbreaker.instances.grades.event-consumer-buffer-size=10
resilience4j.circuitbreaker.instances.grades.sliding-window-type=COUNT_BASED
resilience4j.circuitbreaker.instances.grades.sliding-window-size=5
resilience4j.circuitbreaker.instances.grades.failure-rate-threshold=30
resilience4j.circuitbreaker.instances.grades.wait-duration-in-open-state=5s
resilience4j.circuitbreaker.instances.grades.permitted-number-of-calls-in-half-open-state=3
resilience4j.circuitbreaker.instances.grades.automatic-transition-from-open-to-half-open-enabled=true
```

Struktura projektu

Struktura każdego mikroserwisu, została przygotowana na wzór:



Odpowiednie nazewnictwo wewnątrz projektu pozwoliło w łatwy sposób reprezentować każdą część projektu. Wszystko zostało uporządkowane np. Serwisy w paczce service, kontrolery w paczce controller. Dzięki temu kod jest czysty i przejrzysty oraz każdy programista, który chciałby zmodyfikować projekt jest w stanie odnaleźć się w projekcie.

Aspekt baz danych

Każdy obiekt ma swoje repozytorium, które reprezentuje połączenie obiektu z bazą danych. Logika dla MySQL oraz PostgreSQL jest implementowana dzięki JpaRepository. Dla MongoDB Spring Boot ma swoją własną implementację z dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

MongoDB:

Implementacja obiektu mongodb za pomocą adnotacji:

```
> import ...

18 usages  CzarnaWoda
@Document
@Data
@NoArgsConstructor
public class Subject {

    @MongoId
    private String id;
    private String subjectName;
    private int teacherId;
    private String classId;

    1 usage  create test  CzarnaWoda
    public Subject(String subjectName, int teacherId, String classId){
        this.subjectName = subjectName;
        this.teacherId = teacherId;
        this.classId = classId;
    }
}
```

Document oznacza że jest to obiekt bazy danych, Mongold oznacza pole, które odpowiada za id obiektu, reszta jest automatycznie interpretowana przez spring boota jako pola w bazie danych dla danego obiektu.

```

@Repository
public interface ClassRepository extends MongoRepository<Class,String> {

    1 usage  create test  ⬆ CzarnaWoda
    List<Class> getClassesByTeacherId(int teacherId);

    1 usage  create test  ⬆ CzarnaWoda
    Optional<Class> getClassById(String classId);

    no usages  create test  ⬆ CzarnaWoda
    Optional<Class> getClassByClassName(String className);

    1 usage  create test  ⬆ CzarnaWoda
    boolean existsByClassName(String className);

    1 usage  create test  ⬆ CzarnaWoda
    Optional<Class> getClassByClassCode(String classCode);

    1 usage  create test  ⬆ CzarnaWoda
    boolean existsByClassCode(String classCode);

    1 usage  create test  ⬆ CzarnaWoda
    boolean existsById(String id);

    2 usages  create test  ⬆ CzarnaWoda
    void deleteById(String id);

}

```

Oto jak wyglądałyby zapytania w MongoDB, gdybyś chciał samodzielnie wysłać je bezpośrednio do bazy danych:

1. `getClassesByTeacherId(int teacherId)`:

```
db.class.find({ "teacherId": teacherId });
```

To zapytanie zwraca listę dokumentów, gdzie pole `teacherId` ma wartość równą przekazanemu `teacherId`.

2. `getClassById(String classId)`:

```
db.class.findOne({ "_id": ObjectId(classId) });
```

To zapytanie zwraca pojedynczy dokument, gdzie pole `_id` (standardowy identyfikator dokumentu w MongoDB) ma wartość równą przekazanemu `classId`. Uwaga: w MongoDB identyfikator `ObjectId` musi być przekazany jako typ `ObjectId`.

3. `getClassByClassName(String className)`:

```
db.class.findOne({ "className": className });
```

To zapytanie zwraca pojedynczy dokument, gdzie pole `className` ma wartość równą przekazanemu `className`.

4. `existsByClassName(String className)`:

```
db.class.countDocuments({ "className": className }) > 0;
```

To zapytanie sprawdza, czy istnieje dokument, gdzie pole `className` ma wartość równą przekazanemu `className`.

5. `getClassByClassCode(String classCode)`:

```
db.class.findOne({ "classCode": classCode });
```

To zapytanie zwraca pojedynczy dokument, gdzie pole `classCode` ma wartość równą przekazanemu `classCode`.

6. `existsByClassCode(String classCode)`:

```
db.class.countDocuments({ "classCode": classCode }) > 0;
```

To zapytanie sprawdza, czy istnieje dokument, gdzie pole `classCode` ma wartość równą przekazanemu `classCode`.

7. `existsById(String id)`:

```
db.class.countDocuments({ "_id": ObjectId(id) }) > 0;
```

To zapytanie sprawdza, czy istnieje dokument, gdzie pole `_id` jest równe przekazanemu `id`. Uwaga: w MongoDB identyfikator `ObjectId` musi być przekazany jako typ `ObjectId`.

8. `deleteById(String id)`:


```
db.class.deleteOne({ "_id": ObjectId(id) });
```

To zapytanie usuwa dokument z kolekcji `class`, gdzie pole `_id` jest równe przekazanemu `id`. Uwaga: w MongoDB identyfikator `ObjectId` musi być przekazany jako typ `ObjectId`.

Dla Postgres oraz MySQL używamy `JpaRepository`, które wykonuje zapytania do bazy danych:
Implementacja obiektu w relacyjnych bazach danych:

```
@Entity
@NoArgsConstructor
@Data
public class Grade {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private int grade;
    private int teacherId;
    private int studentId;
    private String subjectId;
    private String teacherName;
    private String studentName;
    private String subjectName;

    1 usage create test 1 CzarnaWoda +1
    public Grade(int grade, int teacherId, int studentId, String subjectId, String teacherName, String studentName, String subjectName){
        this.grade = grade;
        this.teacherId = teacherId;
        this.subjectId = subjectId;
        this.studentId = studentId;
        this.teacherName = teacherName;
        this.studentName = studentName;
        this.subjectName = subjectName;
    }
}
```

Entity oznacza nasz obiekt jest obiektem bazy danych, adnotacja `id` oznacza, które pole reprezentuje `Id`. Dla Postgres oraz MySQL jest to `long`.

Repozytorium:

```

@Repository
public interface NoteRepository extends JpaRepository<Note, Long> {

    1 usage create test 1 CzarnaWoda
    Optional<Note> getNoteById(Long id);

    1 usage create test 1 CzarnaWoda
    List<Note> getNotesByStudentId(int studentId);

    1 usage create test 1 CzarnaWoda
    List<Note> getNotesByTeacherId(int teacherId);

    1 usage create test 1 CzarnaWoda
    @Query("SELECT SUM(n.points) FROM Note n WHERE n.studentId = :studentId")
    Integer getTotalPointsByStudentId(@Param("studentId") int studentId);

    2 usages create test 1 CzarnaWoda
    @Query("SELECT COUNT(n) from Note n where n.studentId = :studentId")
    Integer getTotalNotesAmountByStudentId(@Param("studentId") int studentId);

    no usages create test 1 CzarnaWoda
    @Modifying
    @Query("UPDATE Note n SET n.description = :description, n.points = :points WHERE n.id = :noteId")
    void updateNoteDescriptionPointsByNoteId(@Param("noteId") long noteId, @Param("points") int points, @Param("description") String description);

    1 usage create test 1 CzarnaWoda
    void deleteById(long noteId);
}

```

- **getNoteById(long id):**

SELECT * FROM Note WHERE id = ?;

To zapytanie zwraca pojedynczy wiersz z tabeli Note, gdzie kolumna id jest równa przekazanej wartości id.

- **getNotesByStudentId(int studentId):**

SELECT * FROM Note WHERE studentId = ?;

To zapytanie zwraca wszystkie wiersze z tabeli Note, gdzie kolumna studentId jest równa przekazanej wartości studentId.

- **getNotesByTeacherId(int teacherId):**

SELECT * FROM Note WHERE teacherId = ?;

To zapytanie zwraca wszystkie wiersze z tabeli Note, gdzie kolumna teacherId jest równa przekazanej wartości teacherId.

- **getTotalPointsByStudentId(@Param("studentId") int studentId):**

SELECT SUM(points) FROM Note WHERE studentId = ?;

To zapytanie oblicza sumę wartości kolumny `points` dla wszystkich wierszy, gdzie kolumna `studentId` jest równa przekazanej wartości `studentId`.

- **`getTotalNotesAmountByStudentId(@Param("studentId") int studentId):`**

```
SELECT COUNT(*) FROM Note WHERE studentId = ?;
```

To zapytanie liczy liczbę wierszy w tabeli `Note`, gdzie kolumna `studentId` jest równa przekazanej wartości `studentId`.

- **`updateNoteDescriptionPointsByNoteId(@Param("noteId") long noteId, @Param("points") int points, @Param("description") String description):`**

```
UPDATE Note SET description = ?, points = ? WHERE id = ?;
```

To zapytanie aktualizuje kolumny `description` i `points` dla wiersza w tabeli `Note`, gdzie kolumna `id` jest równa przekazanej wartości `noteId`.

- **`deleteById(long noteId):`**

```
DELETE FROM Note WHERE id = ?;
```

To zapytanie usuwa wiersz z tabeli `Note`, gdzie kolumna `id` jest równa przekazanej wartości `noteId`.

Aby zminimalizować ilość requestów do baz danych, w projekcie powinna być zaimplementowana logika cache. W naszym przypadku nie zaimplementowaliśmy żadnego cache w projekcie z racji na ograniczony czas.

Requesty

Same requesty zostały napisane przy użyciu następującej biblioteki:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Dzięki temu jesteśmy w stanie kontrolować co trafia do bazy danych zanim przejdzie przez endpoint.

Przykładowy record z requestem:

```
public record StudentRegisterRequest(  
  
    1 usage  
    @NotBlank(message = "Name can not be blank")  
    @NotNull(message = "Name can not be null")  
    @Size(min = 6, max = 28, message = "Name must be between 6 and 28 length")  
    String firstName,  
  
    1 usage  
    @NotBlank(message = "Last name can not be blank")  
    @NotNull(message = "Last name can not be null")  
    @Size(min = 6, max = 28, message = "Last name must be between 6 and 28 length")  
    String lastName,  
  
    1 usage  
    @NotBlank(message = "Password can not be blank")  
    @NotNull(message = "Password name can not be null")  
    @Size(min = 6, max = 28, message = "Password name must be between 6 and 28 length")  
    String password,  
  
    2 usages  
    @Pattern(regexp = ".*@.*\\.\\..*")  
    @Size(min = 11, max = 48, message = "Email must be between 11 and 48 length")  
    String email,  
  
    1 usage  
    @NotBlank(message = "Class code is required")  
    String classCode  
)  
{
```

Dzięki adnotacjom takich jak Size jesteśmy w stanie zadeklarować w samym kodzie jaki ma być rozmiar danej zmiennej. Spring boot sam mapuje json body do requesta w endpointzie.

Przykładowy endpoint realizujący request:

```

create test  CzarnaWoda
@PostMapping(@"*/register")
@CircuitBreaker(name = "subjectclass-service", fallbackMethod = "subjectClassFallBackMethod")
public ResponseEntity<HttpResponse> registerStudent(@RequestBody @Valid StudentRegisterRequest registerRequest, BindingResult bindingResult){
    if(bindingResult.hasErrors()){
        return ResponseEntity.status(BAD_REQUEST).body(HttpResponse.builder()
            .httpStatus(BAD_REQUEST)
            .statusCode(BAD_REQUEST.value())
            .message(bindingResult.getAllErrors().get(0).getDefaultMessage()).build());
    }
    if(studentService.existStudentByEmail(registerRequest.email())){
        return ResponseEntity.status(BAD_REQUEST).body(HttpResponse.builder()
            .httpStatus(BAD_REQUEST)
            .statusCode(BAD_REQUEST.value())
            .message("Student already exist!")
            .developerMessage("Student with this email exist in repository")
            .timestamp(dateUtil.getNowDate()).build());
    }else{
        Student student = studentService.createStudent(registerRequest);

        final StudentDTO studentDTO = studentMapper.mapStudentToStudentDTO(student);
        return ResponseEntity.status(OK).body(HttpResponse.builder()
            .httpStatus(OK)
            .statusCode(OK.value())
            .message("Student has been created")
            .data(Map.of("Student", studentDTO)).build());
    }
}

```

DTO oraz Mappery:

Każdy obiekt w naszym projekcie posiada swój odpowiednik DTO (Data Transfer Object). Za pomocą tej logiki jesteśmy w stanie określić po pobraniu obiektu z bazy danych, które konkretnie wartości mają być ‘wyrzucone’ na end-point. Przykładowo dla studenta, nie chcemy wyrzucać jego hasła na ‘zewnątrz’, więc został stworzony obiekt StudentDTO, który mapuje obiekt Student w samym mapperze a następnie wysyła go jako data na endpoint.

StudentDTO:

```

11 usages  👤 CzarnaWoda
7  @data
8  public class StudentDTO implements Serializable {
9      private String firstName;
10     private String lastName;
11
12     private String email;
13
14     private String classId;
15
16
17     /*
18     TODO: classID same as in Student class
19     */
20
21
22
23 }
24

```

StudentMapper:

```

@Component
public class StudentMapper {

    5 usages  create test  👤 CzarnaWoda
    public StudentDTO mapStudentToStudentDTO(Student student){
        StudentDTO studentDTO = new StudentDTO();
        BeanUtils.copyProperties(student, studentDTO);

        return studentDTO;
    }
}

```

Dzięki takiemu rozwiązaniu mamy łatwy sposób mapowania obiektów i ich konkretnych wartości na endpoint.

Przykładowy endpoint z mapperem:

```
@GetMapping("/me")
public ResponseEntity<HttpResponse> me(Authentication authentication){

    Optional<Student> optionalStudent = studentService.getStudentByEmail(authentication.getName());

    if(optionalStudent.isPresent()){
        return ResponseEntity.status(OK).body(HttpResponse.builder()
            .message("Found user data")
            .data(Map.of( "user", studentMapper.mapStudentToStudentAuthenticationDto(optionalStudent.get(), authentication)))
            .build());
    }else{
        Optional<Teacher> optionalTeacher = teacherService.getTeacherByEmail(authentication.getName());

        return optionalTeacher.map(teacher -> ResponseEntity.status(OK).body(HttpResponse.builder()
            .message("Found user data")
            .data(Map.of( "user", teacherMapper.mapTeacherToTeacherAuthenticationDto(teacher, authentication)))
            .build()))
            .orElseGet(() -> ResponseEntity.status(NOT_FOUND).body(
                HttpResponse.builder()
                    .message("User has not be found")
                    .build()
            ));
    }
}
```

Podział mikroserwisów, ze względu na bazy danych:

